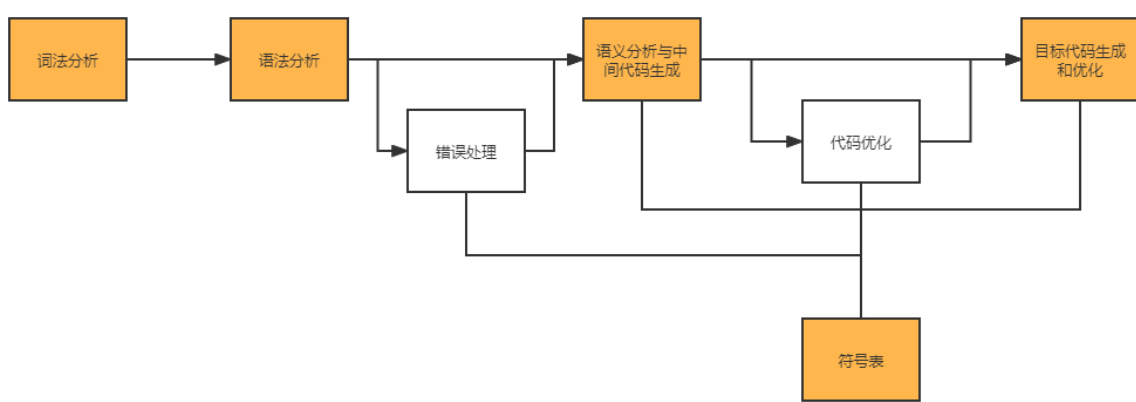


编译器设计文档

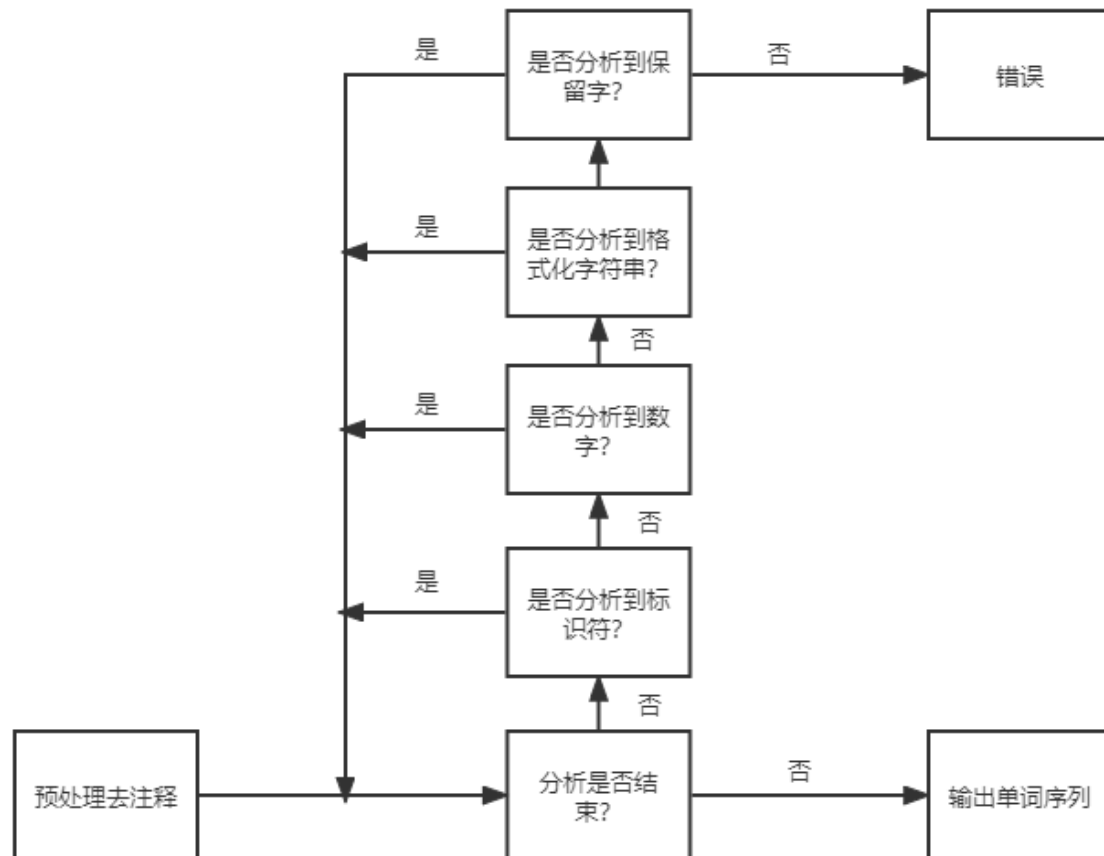
总体架构



编译器从架构上分为7个组成部分，即词法分析、语法分析、语义分析与中间代码生成、代码优化、目标代码生成、错误处理、符号表。

- 源代码从词法分析器输入，将源代码分割为单词序列。
- 单词序列输入语法分析器，输出语法树。
- 语法树经过错误处理部分检查后，如果没有错误的话会直接进入语义分析与中间代码生成器，否则会报错后终止编译。如果能够保证源代码中没有语法错误和语义错误，那么就可以直接将语法树输入语义分析与中间代码生成器。
- 语义分析与中间代码生成器根据输入的语法树，生成中间代码。
- 中间代码可以经过直接生成目标代码，也可以经过优化之后再生成目标代码。
- 在目标代码生成阶段，也可以进行一定程度的优化。

词法分析



编码前设计

将所有单词划分为四类：标识符、数字、格式化字符串、保留字。因此可以在词法分析器中设计四类识别器。这四类单词的开头都有自己的特点：标识符开头是字母或者下划线，数字的开头是数字，格式化字符串的开头是双引号，保留字的话可以一次性读入预估的前缀然后判断是否为保留字，然后再根据这些单词的其它特点，可以设计对应的识别器。因此可以依次调用标识符、数字、格式化字符串、保留字识别器，如果靠前的识别器失败，则调用下一个识别器，如果识别成功，则开始下一次识别循环，直到所有字符已经被读完。

编码后修改

编码前没有考虑到注释的影响，从而造成识别错误，因此在编码过程中加入了一个预处理去注释的过程。

对于保留字的识别，由于一个保留字可能是留一个保留字的前缀，例如 `<相对于<=`，因此在匹配过程中采取了最长匹配的策略。

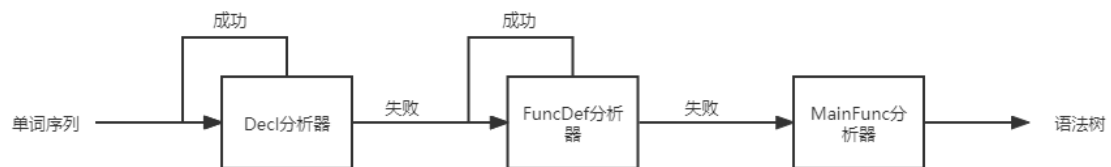
语法分析

编码前设计

语法分析使用递归下降分析法，为每一个非终结符编写了一个分析器。

以 `< CompUnit >` 为例，它的生成式为

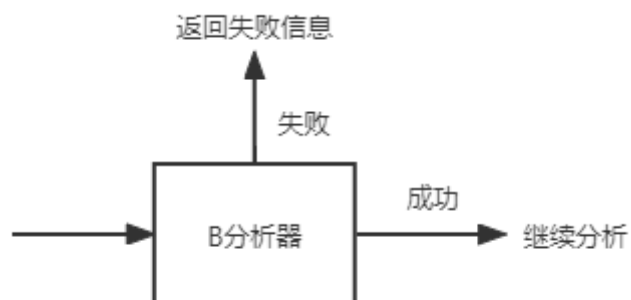
`< CompUnit > → { < Decl > } { < FuncDef > } < MainFuncDef >`，它的分析器的结构如下：



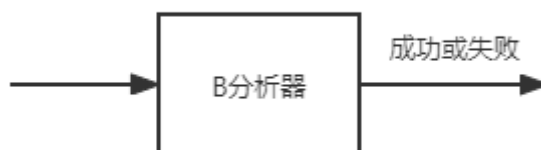
在CompUnit的分析器中，又调用了Decl分析器、FuncDef分析器、MainFunc分析器。

对于生成式中的几种规则，可以按照一定的原则调用分析器，如下：

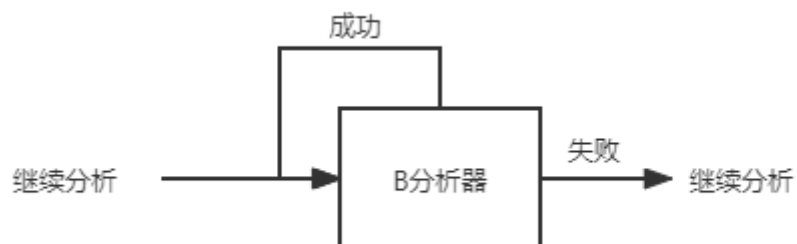
- $A \rightarrow B$



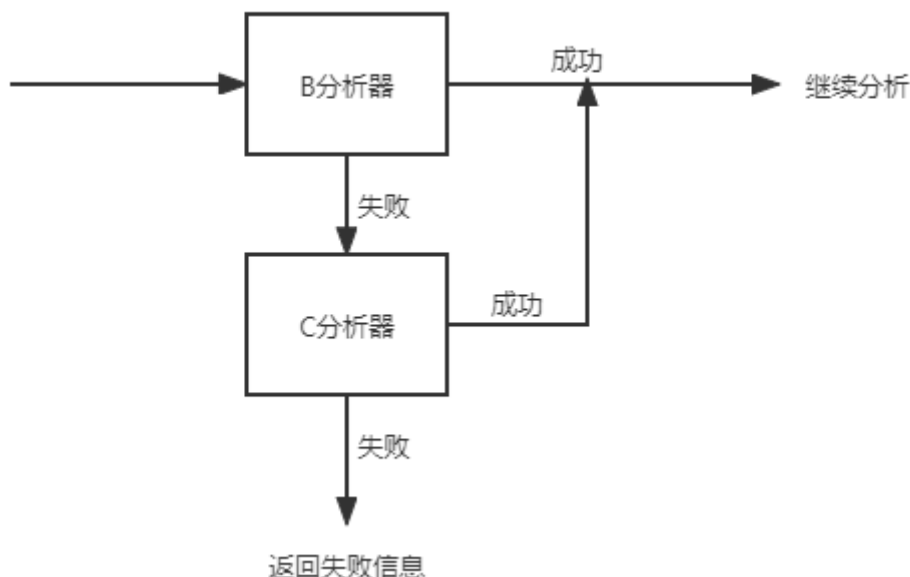
- $A \rightarrow [B]$



- $A \rightarrow \{B\}$



- $A \rightarrow B|C$



按照上述的规则，我对每一个非终结符编写了相应的分析器。如果在某一个语法成分的子成分分析失败，那么失败信息会逐层回传，从而判定该语法成分整体分析失败。在进行完分析之后，只有那些分析成功的语法成分会被保留下来，最终可以得到一棵语法树。

编码后修改

对左递归的生成式进行了修改，例如：

$\langle AddExp \rangle \rightarrow \langle MulExp \rangle \mid \langle AddExp \rangle ('+' \mid '-') \langle MulExp \rangle$

修改为：

$\langle AddExp \rangle \rightarrow \langle MulExp \rangle \mid \langle MulExp \rangle \{ ('+' \mid '-') \langle MulExp \rangle \}$

对所有的左递归生成式进行修改，从而保证了递归下降分析法能够顺利执行。

错误处理

编码前设计

将错误处理拆分为三部分来进行——在词法分析阶段处理的错误、在语法分析阶段处理的错误和需要经过语义分析之后才能处理的错误。

词法分析阶段处理的错误

课程中需要处理的该类错误只有一类，即格式化字符串中出现了非法字符。对于该种错误的处理方案是在词法分析阶段识别格式化字符串的时候检查其中的字符是否符合要求，如果不符合则输出相应错误。

语法分析阶段处理的错误

这部分需要处理的错误是缺少分号 ';' 的错误，缺少右小括号 ')' 的错误，缺少右中括号 ']' 的错误。这三种错误的处理办法是在语法分析阶段进行识别。如果在分析函数中检测到缺少相应的符号，则输出相应的错误，并且补充相应缺少的符号（这样做是为了能够保证语法分析能够继续进行下去，从而能够识别出其他的错误）。

需要经过语义分析之后才能处理的错误

这部分错误分析需要使用之前已经建立好的语法树，同时需要建立栈式符号表。操作符号表的具体策略是：每当进入一个层的时候（一对大括号括起来的部分），就给堆栈增加一个新的层（这个层只是逻辑上的层），内层可以访问外层的变量，而外层不能访问内层的变量。每当离开一个层的时候，则将符号表中当前的层删去。当定义了变量或者函数的时候，则将变量名或者函数名以及相应的属性加入到符号表的栈顶。下面几段讲述对于具体错误的处理方案。

为了找出所有的错误，需要深度优先遍历语法树。当遍历到定义变量或使用变量的语法成分的时候，则根据需求搜索符号表的当前层或者所有层，检查是否有重定义的变量或者没有定义的变量，然后判断是否输出错误。当遍历到调用函数的语法成分的时候，则根据函数名从符号表中搜索出相应的函数以及对应的属性，根据搜索结果判断函数参数的匹配情况，判断是否输出错误。

对于函数返回值是否匹配的错误检测，则在识别到'return'语句或者是在函数结束后进行判断。为此需要从符号表中提取出函数的属性信息，然后进行对照查看是否出错。

对于不能改变常量值的错误进行识别，需要在赋值的时候从符号表中取出相应变量的属性进行对照，然后判断是否输出错误。

对于在不在循环块中使用'break'和'continue'的错误，检测方法是设置一个全局变量用于表示现在是否在循环块中。每当进入或者离开循环块的时候都对这个全局变量进行维护。当识别到'break'或者'continue'的时候都需要检查这个全局变量，然后判断是否出错。

对于 'printf' 变量数和格式化字符串中'%d'的个数不匹配的问题，需要在出现'printf'出现的时候单独进行相应的分析和处理。

编码后修改

对于语法分析中识别出缺少某些字符后的处理，将直接补充相应缺少的符号'的策略修改为'根据缺少该符号后是否仍能识别出合法的语法成分来决定是否补充该字符'。

语义分析与中间代码生成

编码前设计

基本思路是深度优先遍历一遍语法树，对于每种语法成分单独设计分析器，分析器会产生中间代码。

中间代码总共有20种类型，如下表所示：

序号	类型	示例	作用
1	表达式	$t1 = a + 1$	计算
2	函数声明	<code>int func()</code>	声明函数
3	函数块开始	<code>@funcBegin func</code>	标志进入了 <code>func</code> 函数
4	函数块结束	<code>@funcEnd func</code>	标志离开了 <code>func</code> 函数
5	函数形参	<code>para a</code>	<code>a</code> 是形参, 结合函数声明使用
6	实参压栈	<code>push a</code> 或者 <code>push a[0]</code>	<code>a</code> 是实参, 结合函数调用使用
7	函数调用	<code>call func</code>	调用 <code>func</code> 函数
8	函数返回	<code>ret a</code> 或者 <code>ret</code>	函数返回
9	变量声明	<code>var int a</code>	声明一个变量 <code>a</code>
10	常量声明	<code>const int a</code>	声明一个常量 <code>a</code>
11	标签	<code>@label label1</code>	产生一个标签 <code>label1</code> , 用于跳转语句
12	比较跳转	<code>beq #t1 0 label</code>	判断 <code>#t1</code> 是否等于0, 然后决定是否跳转到 <code>label</code>
13	直接跳转	<code>goto label</code>	直接跳转到 <code>label</code> 的位置
14	数组定义	<code>arr int a[4]</code>	声明一个数组 <code>a</code> , 并确定了它的长度
15	数组被传值	<code>a[2] = b</code>	数组被赋值
16	数组传值	<code>b = a[2]</code>	将数组中的某一位置的值赋给另一个变量
17	读入数字	<code>@getint a</code>	将数字读入 <code>a</code>
18	输出	<code>@printf %d a</code> 或 <code>@printf 123</code>	输出值数字或者字符串
19	块开始	<code>@block_begin</code>	块开始
20	块结束	<code>@block_end</code>	块结束

在遍历各个语法成分的过程中, 不断进行符号表的修改, 然后根据符号表信息和当前分析的语法成分的语义信息, 生成相应的中间代码。

编码后设计

上述20个语法成分在编码前并没有考虑完全。

对于型如 $b = a[2] + c$ 这种类型的表达式, 由于"`[]`"本身也是一种运算, 因此考虑转换为 $\#t1 = a[2]$ 和 $b = \#t1 + c$ 这种模式, 便于后续处理, 从而有了第16项。第15项也是基于类似的考虑而产生。

而对于19和20项，在编码前我并没有考虑到大括号对符号表的控制作用，因此并没有加入第19和第20项，而是在进入函数、离开函数以及进入循环、离开循环的时候添加控制项。先前的做法复杂了编码的过程。因此我最终还是加入了19、20，以便以控制符号表的层数。

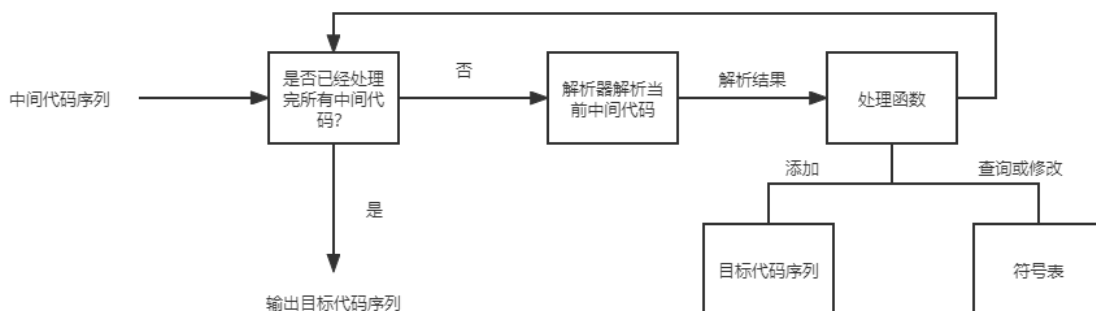
目标代码生成

编码前设计

首先实现一个解析器函数用于判断每一条中间代码在上述20类中的分类，并且可以从中获取参数。

同时实现20个处理函数，分别对应上述20类中间代码，可以根据中间代码的信息修改符号表以及生成目标代码。

然后遍历中间代码，对于每条中间代码，使用解析器进行解析，将解析结果分配给相应的处理函数进行处理。



编码后设计

主体框架并没有改变，只是在实现的过程中修修补补了一些小细节。

值得注意的有这样一点：

实参压栈

实参压栈的时候需要区分传进去的是值还是数组地址。

举一个例子

```
int a = 1;
int b[4];

int func(int a,int b[]){
    return 0;
}

int main(){
    func(a,b);
    return 0;
}
```

对于这段代码，我会生成这样的中间代码：

```
var int a
a = 1
arr int b[4]
int func()
para int a#4
```

```

para int b#5
@func_begin func
ret 0
@func_end func
int main()
@func_begin main
push a
push b[0]
call func
#t0 = RET
ret 0
@func_end main

```

值得注意的是倒数第5行的

```
push b[0]
```

由于我的程序在分析中间代码生成目标代码的过程中并没有保存一个函数形参的类型信息，因此在实参压栈的时候需要主动标注当前传入的是地址还是值。

对于这一行中间代码，我的程序能够清楚识别需要传入地址（因为我约定了[]代表传入地址），这样就能生成这样的正确代码：

```

la $t1,b
sw $t1,xxx($sp)

```

如果中间代码是如下的：

```
push b
```

则会产生这样的错误代码：

```

lw $t1,b
sw $t1,xxx($sp)

```

代码优化

寄存器分配

全局寄存器分配(\$s0-\$s7)

我会在目标代码生成的分析刚刚进入一个函数的时候提前扫描这个函数（不进行修改符号表、生成目标代码处理）的所有中间代码，根据中间代码中的跳转语句划分基本块和生成流图。根据流图进行活跃变量分析，得出每个基本块入口和出口处的活跃变量。注意，活跃变量分析的变量不包括全局变量和临时变量。

接着对于每一个基本块，从块的末尾开始向前扫描，根据变量被使用和被定义的情况修改在每条中间代码处变量的活跃情况。再结合每条中间代码定义变量的情况以及规则“在定义时活跃的其它变量与改变量冲突”构建冲突图。

根据冲突图的结构，使用图着色方法分配全局寄存器，得到一个在函数内部固定了的变量和全局寄存器的对应关系。

这是个很好的结果，当函数内部某处需要使用某变量时，可以查询该对应关系，从而得到应该使用的寄存器（当然也可能该变量没有分配到全局寄存器）。

当然，可以使用DU链使得寄存器的分配更优，但是这样做要么需要修改特定位置的变量名（很麻烦），要么记录变量的位置信息，这两种情况都不如只使用活跃变量分析得到的结果简洁。考虑到实现难度，我最终没有使用DU链进行分析。

临时寄存器分配(\$t0-\$t7)

实现一个临时寄存器池，每当一个变量需要使用临时寄存器的时候，就向临时寄存器池进行申请，由临时寄存器池对改变量分配寄存器。分配情况包含以下三种情况：

- 申请临时寄存器的变量已经占据了一个临时寄存器：无需其它操作，只需要直接返回占据的临时寄存器的名字即可
- 申请临时寄存器的变量没有占据临时寄存器，存在空闲的临时寄存器：将空闲的临时寄存器分配给该变量
- 申请临时寄存器的变量没有占据临时寄存器，不存在空闲的临时寄存器：根据opt算法选择将要分配的临时寄存器，然后对目前占据该临时寄存器的变量，将寄存器中的值写回内存（但并不是完全一定，后面解释），然后将寄存器分配给申请变量。

为了实现第三点，需要在分析刚刚进入一个基本块的时候初始化所有变量的使用顺序。当选择被占据的变量和寄存器时，根据“优先占据最晚被使用或不再被使用的变量”这一原则（也就是opt算法）进行选择。

对于第三点中的写回操作，其实只需要写回那些脏变量。判断一个变量是否为脏的方法是：每当一个变量申请临时寄存器时，都需要告诉临时寄存器池申请后是用来读还是写，如果是写的话临时寄存器池会将变量表示为脏。据此在写回的时候只需要写回脏数据即可。

临时寄存器池还需要在以下时间点进行更新：

- 进入基本块时清空
- 离开基本块时清空，并将所有脏变量写回内存
- 调用函数前清空，并将所有脏变量写回内存

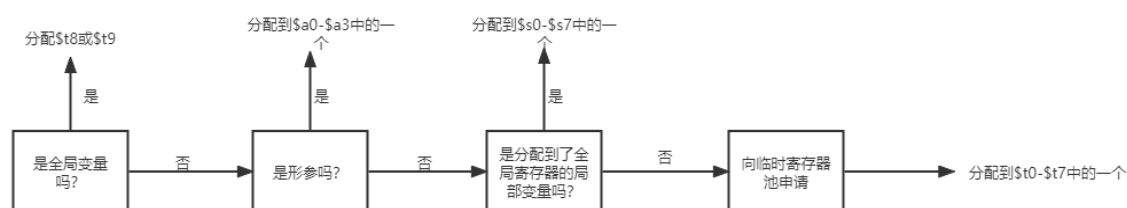
参数寄存器分配 (\$a0-\$a3)

调用函数时前四个参数使用参数寄存器传递。在函数中使用形参的时候类似于分配到了全局寄存器的变量——可以认为形参与参数寄存器——对应。在函数内部调用函数的时候需要提前将参数寄存器中的值存储到对应位置，调用结束之后恢复参数寄存器中的值。

通过以上对参数寄存器的使用，相当于多了四个全局寄存器用于分配，从而提升了效率。

寄存器分配总结

在前面三种分配中，我保留了\$t8和\$t9两个寄存器用于中间运算。在寄存器分配的过程中，我的程序形成了如下框架



运算优化

除法和乘法都是代价较大的运算。如果能够将除法和乘法转化为等价的加减和移位运算，就能大大降低系统消耗。

除法优化

主体思路是将除法优化为乘法和移位指令，以削弱运算强度。

该优化只针对除数为常数的情况：对于除法 $\frac{n}{d}$ ，其中 d 为已知常数，可以考虑将运算转化为 $\frac{m * n}{2^{32+l}}$ ，从而将一次除法转化为了一次乘法和一次移位。

上式中， m 满足 $2^{m+l} \leq m * d \leq 2^{32+l} + 2^l$ ，其中 l 理论上可以任意选取

因此给定 d ，编译器就可以提前计算出 m 和 l ，从而削弱运算强度。

考虑到机器的位数问题， m 和 l 的选取有一定的技巧性，详情见参考文献[1]，这里不多展开。

当然，如果 d 直接就是 2 的次幂，那么除法可以直接简化为一条移位指令。

对于除数已知取余运算，同样可以采取上述优化，相较于除法只是增加了一条乘法指令和减法指令。

乘法优化

主体思路是将乘法转化为移位操作。

该优化只针对两个因子中一个为常数的情况。由于竞速实验中乘法指令的消耗为 3，但实际上完成这一操作需要使用一次 li （用于将常数加载到寄存器），一次 $mult$ 和一次 $mflo$ （加载结果），总共的消耗为 5，因此只要因子符合 $2^m + 2^n$ 或者 $2^m - 2^n$ 的形式，就可以将乘法转化为两次移位和一次加法（或减法），消耗为 3，拥有正向收益。

窥孔优化

窥孔优化是在较小的范围内对代码进行的优化，其实也是对生成代码的不足的弥补。我主要做了以下两种窥孔优化：

删除多余跳转指令

如果一条直接跳转指令的下一行就是跳转标签，那么就可以删除这条跳转指令。如下：

```
xxx1  
j label  
nop  
label:  
xxx2
```

可以转化为：

```
xxx1  
label:  
xxx2
```

合并小范围的冗余表达式

我将类似以下的中间代码进行了合并：

```
#t1 = a + b  
c = #t1
```

转化为：

```
c = a + b
```

参考文献

[1] Granlund, Torbjörn, and Peter L. Montgomery. "Division by invariant integers using multiplication." *Proceedings of the ACM SIGPLAN 1994 conference on Programming language design and implementation*. 1994.